Loan Approval Prediction Final Report: Predict if a loan will be approved or not

Hannah Brown, Karley Keel, Allie Santos

Summary

For this project, we used the Kaggle Loan Prediction dataset to predict whether a loan application will be approved or not. We used binary classification because the target variable had two outcomes: approved or not approved. We wanted our machine learning model to do the cleaning, training, learning, and calculation.

So, we use Logistic Regression, support vector machines, and Decision Tree machine learning models. SVM had the highest ROC/AUC score at 0.7497, while Logistic Regression produced the highest average 5-fold cross validation accuracy at 0.8105. Logistic Regression and SVM both reached a 0.7886 test accuracy, 0.7596 precision, 0.9875 recall, and 0.858 seven F1 score. The Decision Tree model performed the lower overall condition. Its accuracy was only 0.7154, and its Roc slash AUC was only 0.6737.

Based on the data provided, we concluded that the Decision Tree was the least efficient but compared to Logistic Regression and SVM. Logistic Regression performed better on cross-validation, and it was more stable, but SVM had the best ROC/AUC which separated the classes best. However, SVM was less interpretable. Logistic Regression had a slightly lower ROC/AUC, and the Decision Tree was a little bit easier to understand, but overall, its performance was the worst of the three.

Introduction and Problem Statement

Banks tend to make money through lending and investing, but every loan also creates a risk. If a customer defaults on the loan, the bank may lose money, and that's a liability. With our model, we can determine whether a bank should approve or deny a loan based on previous approval data to make loan decisions quicker and easier for banks.

Our target variable in the data set is loan status. In the raw data, loan status was recorded as Y or N. However, whenever we trained the model, we converted this to a binary number 0 or 1. Approved loans were mapped to 1 and disapproved loans were mapped to 0. Many of the input features were gender and marital status, dependents, education, employment, applicant income, loan amount, loan term, credit history, and property area. We didn't manually assign the weights to the features. They were assigned automatically by the model during training.

Challenges

The biggest challenges in this topic were defining a stable model across diverse data. We do not want false approvals or false denials because False approvals caused the bank to approve customers who should not receive a loan, and false denials cause a bank to deny people's credit who may have qualified. That’s why we use multiple testing methods. Our results were surprising because they varied depending on the evaluation.

Dataset

Our dataset is the Kaggle Loan Prediction Problem Dataset. The data set contains 614 loan applications and 13 columns. After loading the data, we inspected the columns, filled the missing values, and determined the target variables:

Item	Value
Total rows	614
Original columns	13
Target variable	Loan_Status
Approved loans	422
Not approved loans	192
Approved percentage	68.73%
Not approved percentage	31.27%

This is a demonstration of the data set, and it shows that there were more approved loans than loans that were not approved. The dataset could be challenging for the machine learning models overall, which may lean towards approving loans.

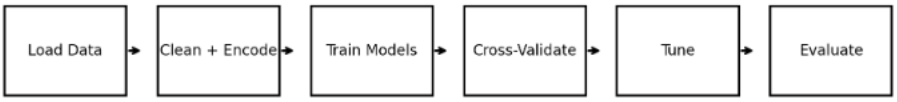
Data Cleaning/Preprocessing

We turned the raw data set into a format that the machine learning model could use. The Loan ID column was removed because it was just an identifier, and it did not provide meaningful predictive information. The target column was converted from Y/N to 1/0. Indicating the yes or no values. Missing values were filled in before the model was trained.

Step	What was done
Drop ID	Removed Loan_ID
Target encoding	Mapped Y to 1 and N to 0
Missing categorical values	Filled using the mode
Missing numerical values	Filled using the mean
Categorical encoding	Used LabelEncoder
Train/test split	80% training and 20% testing
Scaling	Used StandardScaler

We used scaling because Logistic Regression and SVM could be affected by featuring magnitude. The scaler was fit on the training data and then applied to the test data.

Model Workflow Diagram



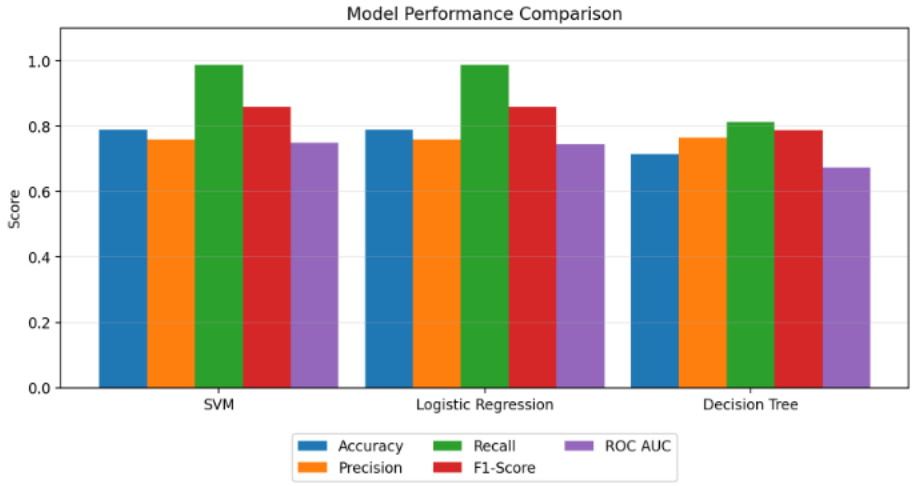
Results

We tested three models. One of the models was Logistic Regression, the other model was SVM, and the other model was a Decision Tree. Logistic Regression was the baseline model, SVM was used to find the separation boundary between classes, and Decision Tree was used because it was easier to understand and interpret.

Model	Reason for using it	Main limitation
Logistic Regression	Fast, interpretable baseline for binary classification	May miss nonlinear patterns
SVM	Strong classifier that can separate classes with a boundary	Less interpretable than Logistic Regression
Decision Tree	Easy-to-understand if/then decision structure	Can overfit or generalize poorly without tuning

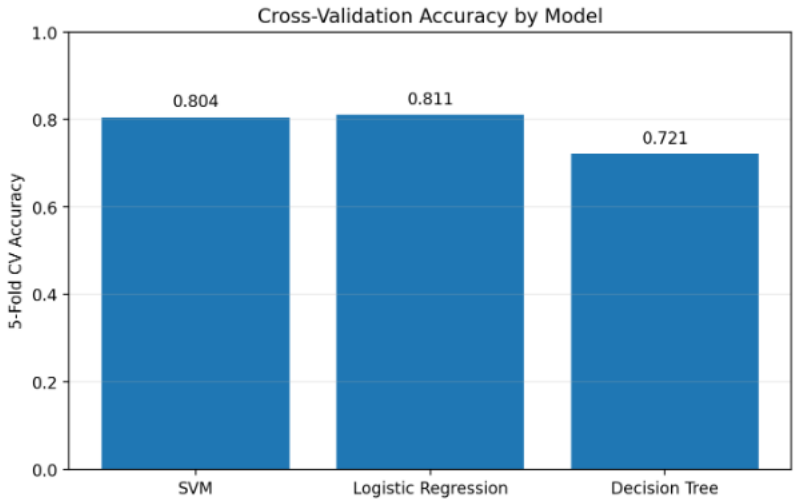
The results from the model show that SVM and Logistic Regression performed the best. They perform better than Decision Trees by a long shot. However, when it comes to comparing both Logistic Regression and SVM, things become a little bit more complicated. Logistic Regression performs better on some tests, and SVM does better on others. This indicates that the original model being slightly skewed towards approval. Using multiple testing methods can create something where different models perform well on different tests.

Model	Accuracy	Precision	Recall	F1-score	ROC AUC	CV Accuracy	Train Time (s)
SVM	0.7886	0.7596	0.9875	0.8587	0.7497	0.8044	0.0250
Logistic Regression	0.7886	0.7596	0.9875	0.8587	0.7445	0.8105	0.0016
Decision Tree	0.7154	0.7647	0.8125	0.7879	0.6737	0.7210	0.0034



Comparison of accuracy, precision, recall, F1-score, and ROC AUC.

For cross validation, we used the 5-fold cross validation to check model stability. This means that the training debt was split into five parts, and each part was used as a validation fold once. Using this average score across folds gives a better idea of how stable the model is than just using a single split. Logistic Regression had the highest cross-validation accuracy at 0.8105. SVM was a close second at 0.8044. Decision Tree had the lowest cross validation accuracy at 0.7210. So according to this testing, Logistic Regression had more stable choices.



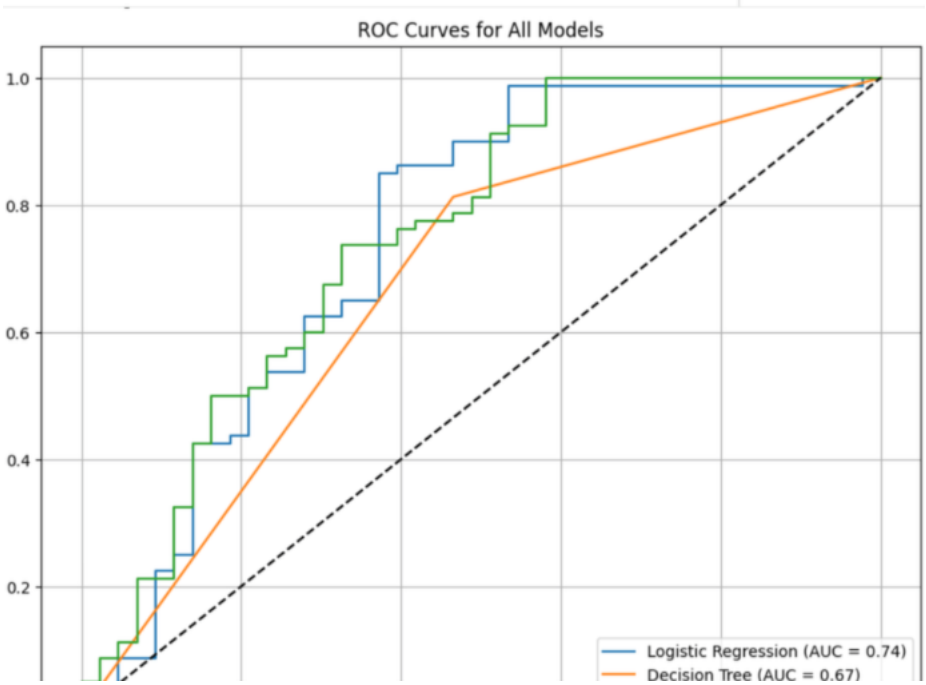
Cross validation Accuracy by model

Our final model check was the confusion matrix. The model approved 79 applications and rejected 18 applications. However, it incorrectly approved 25 applications and rejected 1 application. This shows that class 1 approved loans had a 99% recall. That means the model found almost all approved applications. However, Class 0 of not approved loans had a recall of only 42%, meaning the model struggled to correctly identify rejected applications. We discussed

that the original data set had a slight lean towards approving loans. As a result, the model, in practical terms, leaned towards predicting approval.

Class	Precision	Recall	F1-score	Support
0: Not approved	0.9500	0.4200	0.5800	43
1: Approved	0.7600	0.9900	0.8600	80
Accuracy			0.7900	123
Macro Avg	0.8500	0.7000	0.7200	123
Weighted Avg	0.8300	0.7900	0.7600	123

The Roc curve demonstrates the tradeoff between a true positive rate and a false positive rate. The AUC scores summarized how well the model separates the approved and not approved loans. SVM had the highest Roc AUC score at 75%. Logistic Regression was close at 74%. Decision Tree had the lowest and it was 67%. We changed this into percentage just to be a little bit easier to understand. But the raw numbers were for SVM, 0.7497, Logistic Regression, 0.7445, and Decision Tree 0.6737. But. But as we can see here on this test, SVM performs better.

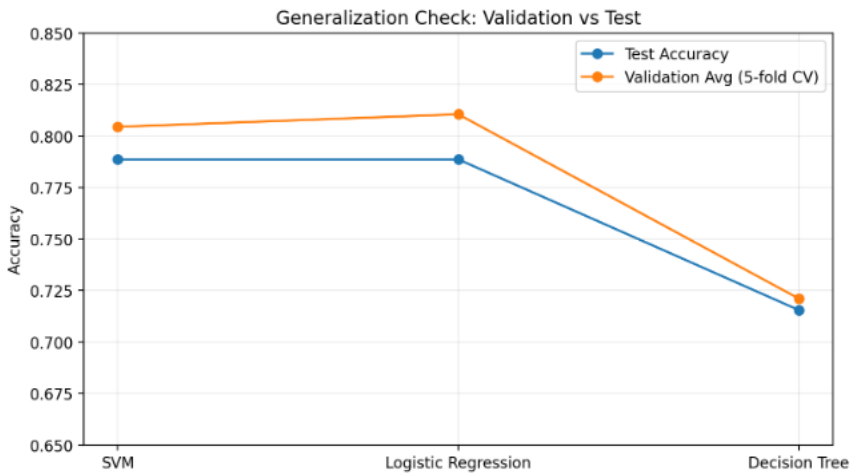


ROC curves for all three models

Underfitting/Overfitting/ Runtime

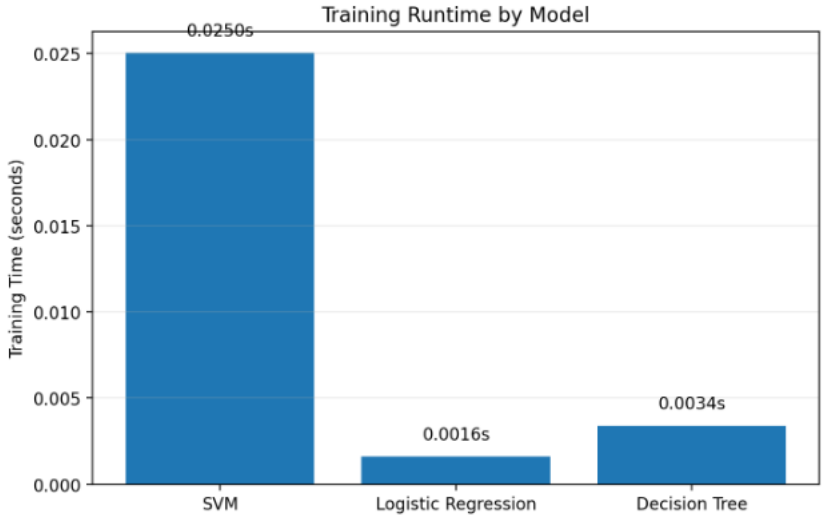
This project used classic machine learning models instead of neural networks. The project checked generalization by comparing final Test performance with cross and predation performance. Logistic Regression and SVM had validation and test results that were close to each other, which suggests they were better than Decision Trees. These test scores indicate an

incredibly low chance of overfitting slash underfitting for SVM and Logistic Regression, but a slightly higher chance of overfitting, for Decision Tree.



Validation average compared with final test accuracy.

The runtime for a Logistic Regression was the fastest of all three. It took about 0.0016 seconds. The Decision Tree took about 0.0034 seconds. SVM was the slowest at about 0.0250 seconds. SVM was a bit slower overall, which could run into problems if the data set were a lot larger.



Training runtime by model.

Team Contributions

Team member	Contribution
Hannah Brown	Team leader, organization, deadlines, final integration
Allie Santos	Feature and label analysis, preprocessing support
Karley Keel	Algorithm design, model comparison, evaluation metrics

Whole team	Reviewed results, finalized slides, and contributed to the report

Conclusion

Some future improvements that are suggested for this project would be to use an additional dataset that may slightly lean more so towards denying loans and another dataset that is more neutral. That way, the models may be more accurate when it comes to weighing the different featurs. The loan Approval project successfully built and evaluated a machine learning workflow or binary classification.

We encoded, cleaned, split, scaled, modeled, cross validated and evaluated using multiple metrics. Decision Tree performed the weakest overall, and SVM and Logistic Regression were two of the strongest models. However, on further testing, SVM and Logistic Regression performed differently on different testing, but the differences were very subtle. The main major difference was that SVM had a slower runtime than Logistic Regression. Because other forms of testing SVM performed better in Logistic Regression performed better across others, However, the differences were very subtle.

The final model results were not perfect. The model was strong at detecting approved loans but weaker at detecting non-approved loans. It had plenty of false positives, meaning it would approve loans that were not meant to be approved. For a real bank this matters because false approvals could increase risk. However, we can mitigate this by utilizing an additional data set to train the model on, and this data set would include rejected thumbs.

Another most important lesson from this project is that model evaluation Can change depending on the type of model. Sometimes two different models can perform well on certain evaluations but perform poorly on others. And sometimes the model that performs well is different depending on the evaluation. Therefore, we used multiple different evaluations including cross validation, confusion matrix, Roc slash AUC, runtime, and interpretation.